

A Compact Unicode Property Database for Text Shaping

Design rationale, runtime API, extension model, and the path toward page-oriented execution

William A. Adams
September 2026

Design position

Unicode is treated here as a compact factual substrate for text processing. The database answers stable questions about code points. Script-specific classification and shaping semantics live above it, where they can evolve independently.

1. Why I built this

I started this work because I wanted a Unicode layer that I could actually reason about while building a text engine. The usual alternatives are capable, but they tend to arrive as part of a much larger stack. For SVG Buddy I wanted something narrower: a database I could generate from the UCD, validate exhaustively, mmap or attach cheaply, and query without dragging policy into the storage layer.

The distinction between facts and behavior matters. Unicode tells me that a code point has a Script, a General_Category, a Canonical_Combining_Class, an Indic_Syllabic_Category, a Joining_Type, and so on. Those are facts. Whether that combination becomes a consonant, repha candidate, medial mark, joining participant, or some project-specific ScriptItemKindId is a separate question. I want that question answered by the shaping layer, not frozen into the database format.

That leads to a deliberately modest database design. The persistent structures are mostly page tables, small records, deduplicated pools, and semantic registries. The generator is allowed to be simple and somewhat wasteful. The runtime is not.

Facts below, shaping semantics above

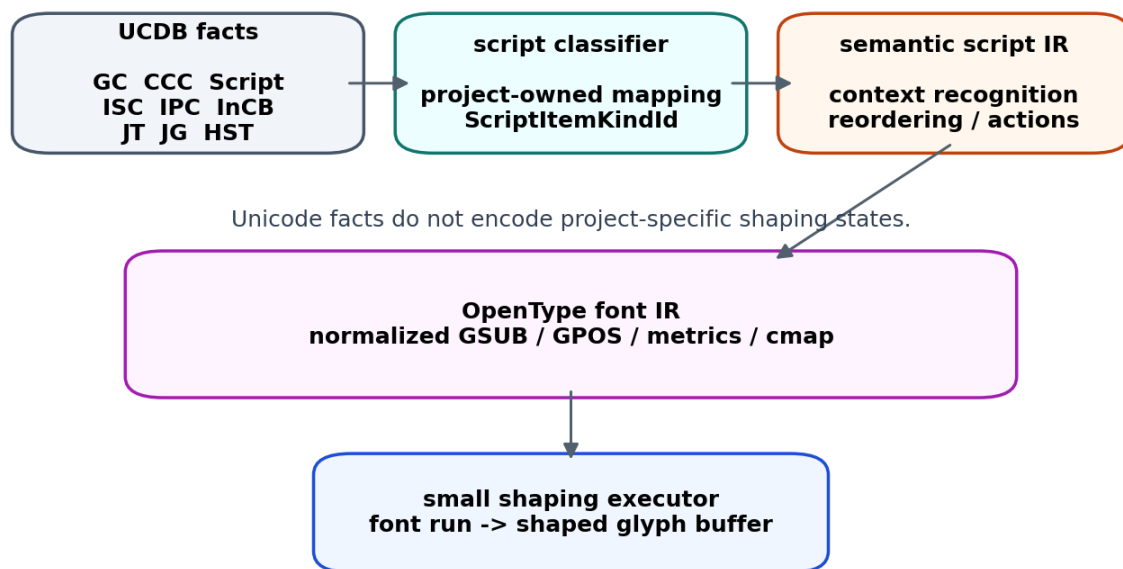


Figure 1. The database supplies Unicode facts; script classification and semantic IR remain separate.

2. Design goals

The design has been driven by a short set of practical goals:

- Fast arbitrary lookup by Unicode scalar value.
- Compact immutable runtime data that can be attached directly to a serialized image.
- One geometry shared by many properties, so adding a property does not require inventing a new container.
- Deduplication at page and root level, including cheap representation of uniform regions.
- Generator-side simplicity: parse into dense mutable builders, then canonicalize once.
- Strong validation: malformed tables, invalid semantic values, duplicate registrations, and incomplete required property sets should fail at attach time.
- Project-owned semantics above the database rather than inside it.
- A clean evolutionary path toward page-oriented execution for shaping workloads that consume several properties together.

A recurring rule

I optimize the runtime representation first. Build-time memory is cheap and short-lived; a branch, cache miss, pointer chase, or extra resident page in the shaping hot path is paid over and over.

3. The Unicode address space as storage geometry

The current containers use a two-level page geometry over the Unicode scalar address space. The important constants are small enough to keep in your head:

Level	Geometry	Role
Unicode space	0x000000 - 0x10FFFF	1,114,112 positions
Leaf page	1024 code points	One VALUE8 byte page or one binary bit page
Master region	32 leaf pages	32,768 code points
Root	34 master regions	Covers the complete Unicode address space

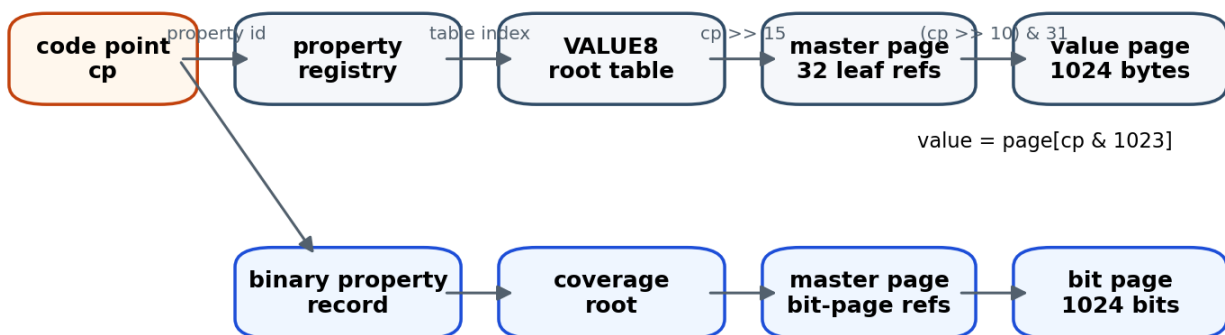
For a code point `cp`, the decomposition is straightforward:

```
master = cp >> 15
leaf   = (cp >> 10) & 31
offset = cp & 1023
```

That same page shape is useful for both binary properties and byte-valued properties, while still allowing each property to deduplicate independently according to its own boundaries.

Current property-major lookup geometry

One semantic property chooses one root table; the code point then walks the shared page hierarchy.



The storage geometry is shared; semantic meaning lives above it in registries and typed accessors.

Figure 2. Current property-major lookup. SET and VALUE8 properties share the same logical page geometry.

4. Binary properties: SET storage

A binary Unicode property answers a membership question: does this code point belong to this set? Examples in the current database include `Extended_Pictographic` and `Default_Ignorable_Code_Point`. Scripts also retain per-script coverage sets because they are useful for discovery, filtering, and coverage operations even though Script has a dense `VALUE8` lookup as well.

The persistent SET form is a root that identifies master pages, each master identifies leaf bit pages, and each leaf holds 1024 membership bits. The pool is shared, so identical pages collapse. Uniform empty/full regions can be represented compactly rather than materialized as ordinary pages.

The consequence is useful beyond raw size: a binary property remains a real set. I can intersect it with font coverage, use it in filtering pipelines, or enumerate coverage without pretending it is merely a byte-valued classifier.

5. VALUE8 properties: one byte of semantic fact

Most shaping properties are small enumerations. A byte is enough for `General_Category`, `Bidi_Class`, `Script index`, `Grapheme_Cluster_Break`, `ISC`, `IPC`, `Joining_Type`, `Joining_Group`, `Hangul_Syllable_Type`, and similar facts. Those properties use the `VALUE8` container.

```
struct UnicodeValuePage8 {
    uint8_t values[1024];
};

struct UnicodeValueMasterPage8 {
    UnicodeValuePage8Ref sub[32];
};

struct UnicodeValueTable8Data {
    UnicodeValueMasterPage8Ref masters[34];
};
```

The builder is intentionally much less clever: it is a dense 1,114,112-byte array. Parsers set values or ranges directly. Finalization turns that dense image into the canonical shared-page representation. That division has made the parsers easier to read and much harder to get subtly wrong.

Uniform values are important

Many Unicode properties are overwhelmingly their default value. The `VALUE8` reference space reserves an inline encoding for uniform byte values, so a default-filled page or master region does not need a physical page allocation.

6. Semantic registry and validation metadata

The original implementation accumulated one runtime member per `VALUE8` property. That worked, but every new property required another member, another reset path, another validator branch, and another opportunity for the property IDs and tables to drift apart. I replaced that with a semantic registry keyed by `UnicodeValueProperty8`.

```
UnicodeValueTable8Index table =
    mValuePropertyTables8.get(UnicodeValueProperty8JoiningType);

if (table == kUnicodeValueTable8IndexInvalid)
    return UnicodeJoiningType::NonJoining;

return static_cast<UnicodeJoiningType>(
    valueTable8(table).value(cp));
```

A companion metadata table says whether a property is required and how its stored values should be validated. Most properties use a fixed maximum derived from the enum count; Script is the interesting exception because its maximum is tied to the number of Script records in that database image.

This turns attach-time validation into data-driven policy instead of a growing switch statement. Adding another ordinary VALUE8 property no longer requires new runtime state.

7. What is currently in the database

The current database has reached the point where it contains the Unicode facts I need for script categorization and the front half of shaping.

Property	Kind	Default / zero	Primary use
General_Category	VALUE8	Unassigned	basic scalar class
Canonical_Combining_Class	VALUE8	0	normalization, mark behavior
Bidi_Class	VALUE8	property-defined	Unicode bidi
Grapheme_Cluster_Break	VALUE8	Other	grapheme segmentation
Indic_Conjunct_Break	VALUE8	None	Indic conjunct behavior
Script	VALUE8 + SETs	Unknown	itemization and script coverage
Indic_Syllabic_Category	VALUE8	Other	Indic character roles
Indic_Positional_Category	VALUE8	NotApplicable	Indic mark/vowel position
Joining_Type	VALUE8	NonJoining	Arabic-family joining behavior
Joining_Group	VALUE8	NoJoiningGroup	joining shape family
Hangul_Syllable_Type	VALUE8	NotApplicable	Jamo/Hangul composition class
Extended_Pictographic	SET	false	emoji/grapheme behavior
Default_Ignorable_Code_Point	SET	false	text processing / filtering

8. Runtime API

The API is intentionally boring. Callers should not know about master pages, leaf references, inline-uniform encoding, or deduplication. They ask semantic questions.

```

UnicodeGeneralCategory gc = database.generalCategory(cp);
uint8_t ccc = database.combiningClass(cp);
UnicodeScriptIndex script = database.script(cp);

UnicodeIndicSyllabicCategory isc = database.indicSyllabicCategory(cp);
UnicodeIndicPositionalCategory ipc = database.indicPositionalCategory(cp);
UnicodeIndicConjunctBreak incb = database.indicConjunctBreak(cp);

UnicodeJoiningType jt = database.joiningType(cp);
UnicodeJoiningGroup jg = database.joiningGroup(cp);

UnicodeHangulSyllableType hst = database.hangulSyllableType(cp);

```

Presence checks exist where useful:

```

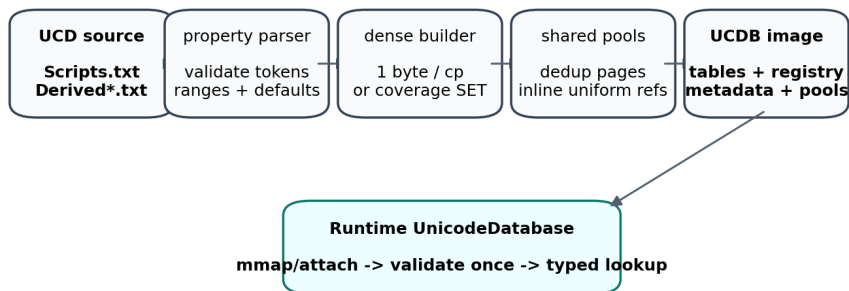
database.hasIndicSyllabicCategory();
database.hasIndicPositionalCategory();
database.hasJoiningType();
database.hasJoiningGroup();
database.hasHangulSyllableType();

```

Typed accessors return the Unicode default for an absent table or an out-of-range code point where that is the established API contract. This keeps classification code uncluttered while attach-time validation still ensures that a production database contains the required tables.

9. Build-time architecture

Build-time pipeline: simple mutable builders, compact immutable runtime



Generator complexity is intentionally disposable; runtime state is compact, validated, immutable, and pointer-light.

Figure 3. The generator favors simple mutable construction; finalization produces the compact immutable form.

For ordinary VALUE8 properties the generator lifecycle is now generic: load the UCD source, create a dense UnicodeValueTable8Builder, invoke the property-specific parser, finalize into the shared VALUE8 pool, add the root table, and register the semantic property ID. The parser retains responsibility for syntax, range overlap, default values, and token-to-enum mapping.

Script remains a justified special case. Parsing Scripts.txt builds both per-Script coverage records and the dense code-point-to-Script-index VALUE8 table. Those two views must share exactly the same Script indices, so they are finalized as one coordinated operation rather than forced through the generic property helper.

10. Extending the database with a binary property

Adding a binary property should be mechanical. The exact parser varies because UCD files vary, but the storage path should not.

1. Define or reuse the semantic property identifier used by the database record.
2. Parse the authoritative UCD source into a UnicodeCoverageBuilder. Initialize the builder empty; add each explicit range; reject malformed or overlapping data when the property requires it.

3. Finalize the coverage into the shared SET page pool.
4. Add the resulting coverage root to the database and register the semantic property record.
5. Expose a typed or named runtime membership query if the property is important enough to deserve one.
6. Add an exhaustive or source-derived test so the serialized database is checked independently of the generator path.

```
UnicodeCoverageBuilder coverage;
// parser: coverage.addRange(first, last);

UnicodeCoverageData data{};
coverage.finalize(database.pagePool(), data);

UnicodeCoverageIndex index;
database.addCoverage(data, index);
// register semantic property -> index
```

11. Extending the database with a VALUE8 property

The VALUE8 path is now even more regular. For an ordinary enumerated Unicode property, the extension recipe is:

7. Define a uint8_t enum. Whenever practical, give Unicode's documented default value numeric value zero.
8. Add a UnicodeValueProperty8 semantic ID and advance UnicodeValueProperty8MAX.
9. Add one record to kUnicodeValueProperty8Info, normally required=true and StaticMaximum with enumCount - 1.
10. Write a property-specific parser that clears the dense builder to the Unicode default, decodes source tokens, assigns explicit ranges, and reports useful counts.
11. Add one buildValueProperty8() block to the generator.
12. Add hasX() and a typed accessor to UnicodeDatabase.
13. Run an exhaustive source-to-runtime round-trip comparison across all 1,114,112 positions.

```
if (!buildValueProperty8(
    database, ucdRoot,
    "IndicPositionalCategory.txt",
    "Indic_Positional_Category",
    UnicodeValueProperty8IndicPositionalCategory,
    result, parseFn, errorFn))
{
    return false;
}
```

Do not infer file syntax from property names

Several UCD files store short aliases in the data rows even when comments use long names. DerivedJoiningType.txt, for example, stores C/D/L/R/T rather than Join_Causing/Dual_Joining/etc.; HangulSyllableType.txt stores L/V/T/LV/LVT. The parser should match the actual source rows.

12. Validation: prove the serialized path, not just the parser

The most useful tests in this project independently parse the authoritative Unicode source and compare the result against the runtime database accessor for every Unicode code point. This catches much more than a parser smoke test: it exercises parsing, default filling, finalization, page deduplication, serialization, database attach, semantic registry resolution, page lookup, and the typed accessor.

The recent shaping-property test compares IPC, Joining_Type, Joining_Group, and HST over the complete address space: $4 \times 1,114,112 = 4,456,448$ runtime comparisons. ISC has its own exhaustive test. All of those comparisons currently pass.

Property	Source ranges	Explicit code points	Defaulted
Indic_Syllabic_Category	967	4,856	1,109,256
Indic_Positional_Category	653	1,299	1,112,813
Joining_Type	542	3,004	1,111,108
Joining_Group	217	356	1,113,756
Hangul_Syllable_Type	804	11,529	1,102,583

13. Why the database stops at facts

Facts below, shaping semantics above

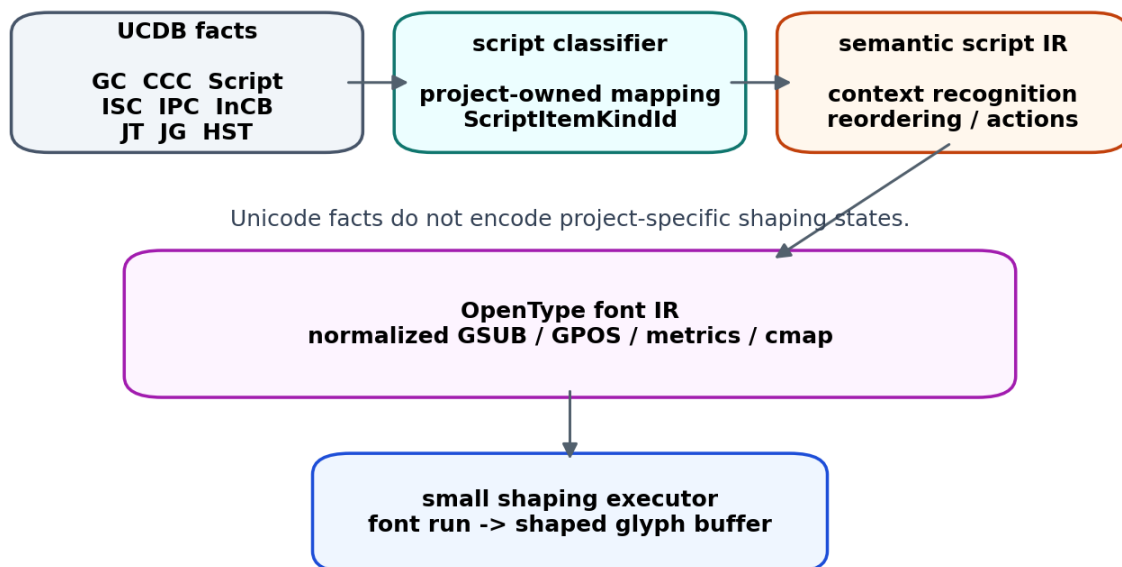


Figure 4. Unicode facts are stable inputs; script classification and contextual behavior remain project-owned.

It is tempting to push more shaping interpretation into the database because the data is already there. I think that is the wrong boundary. ISC and IPC are facts. A Devanagari classifier deciding that a particular combination maps to a project-owned consonant or matra category is policy. Arabic joining analysis using JT and JG is policy. Context recognition that turns several classified items into a reordering or substitution action is even more clearly policy.

Keeping those layers separate gives me two freedoms. First, the UCDB can serve other text APIs besides my shaper. Second, the shaping model can become progressively more semantic - including a script IR - without requiring another Unicode database format every time I learn something about the script engine.

14. Performance model today

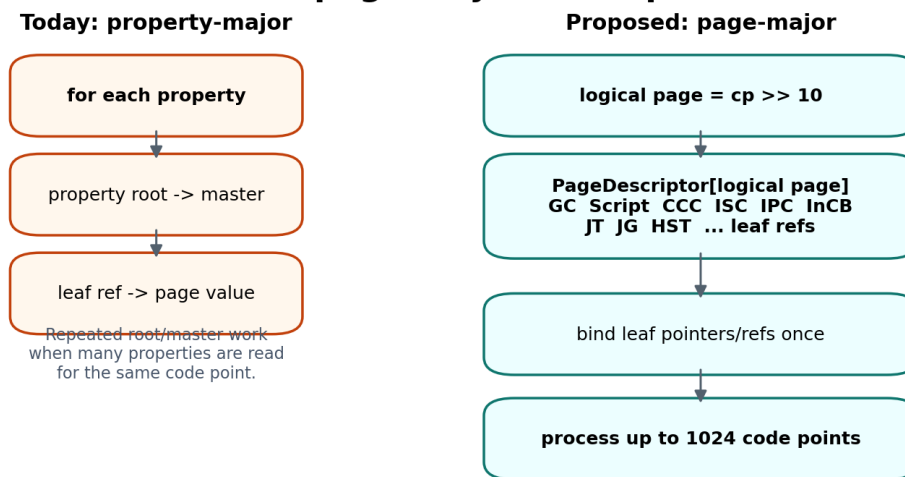
The current lookup model is property-major. A call such as `joiningType(cp)` first resolves the semantic property to its `VALUE8` root, then uses `cp` to reach the master and leaf. This is a good general-purpose API: simple, independent, and cheap enough for arbitrary queries.

Shaping, however, is not an arbitrary-query workload. A script classifier may read General_Category, CCC, Script, ISC, IPC, InCB, JT, JG, and HST for the same stream of code points. The individual tables have different boundaries and should remain independently deduplicated, but the repeated root/master navigation becomes increasingly visible once several properties are consumed together.

This observation motivates the next layer rather than invalidating the current one.

15. Future design: a page-oriented index

Future direction: page-major descriptors over the existing pools



The page-major layer is an index, not a replacement: existing deduplicated SET and VALUE8 pools remain canonical storage.

Figure 5. A page-major descriptor can bind the property leaves needed by a workload while leaving the existing pools untouched.

There are 1,088 logical 1024-code-point pages in the Unicode address space. That is small enough to consider a page descriptor table indexed directly by `cp >> 10`. A descriptor does not need to copy property data. It can simply identify the already-deduplicated leaves for the properties a runtime program cares about.

```
struct UnicodePageDescriptor {
    UnicodeValuePage8Ref generalCategory;
    UnicodeValuePage8Ref script;
    UnicodeValuePage8Ref combiningClass;
    UnicodeValuePage8Ref indicSyllabicCategory;
    UnicodeValuePage8Ref indicPositionalCategory;
    UnicodeValuePage8Ref indicConjunctBreak;
    UnicodeValuePage8Ref joiningType;
    UnicodeValuePage8Ref joiningGroup;
    UnicodeValuePage8Ref hangulSyllableType;
};
```

That exact structure is illustrative, not final. A real implementation may be generated from a ScriptProgram or property mask so that Latin does not pay for Indic fields and an Arabic classifier does not bind Hangul. The key idea is that page transition becomes the expensive event. Once a run is inside one logical page, property access becomes page-local.

This is an index, not a new database

The existing SET and VALUE8 pools remain the canonical storage and continue to support arbitrary accessors. A page-oriented descriptor layer is a workload accelerator that can be built from those same roots.

16. Page binding and script programs

The page-oriented idea becomes more interesting when paired with the semantic script IR work. A script program already knows which Unicode facts it needs. That means the program can declare a property dependency set, and a run cursor can bind those leaves whenever `cp >> 10` changes.

```
ScriptProgram Devanagari:
  requires { GC, CCC, Script, ISC, IPC, InCB }

ScriptProgram Arabic:
  requires { GC, Script, JT, JG }

ScriptProgram Hangul:
  requires { Script, HST }
```

The result is dynamically composable without turning the database into one giant fixed record per code point. Properties keep their independent deduplication behavior, while a hot loop gains something closer to a structure-of-arrays page cache.

17. Longer-term possibilities

I do not think the page descriptor is necessarily the end of the design. Once the shaping system has semantic IR, a few additional optimizations become possible without changing the public Unicode API:

- Precompute page descriptors only for property combinations actually used by installed script programs.
- Reorder or pack descriptor fields according to measured workloads rather than guessing one universal layout.
- Generate specialized classification code for properties whose decision structure is smaller than their table representation.
- Build mmap-ready optimized representations from a corpus-driven compiler while retaining the canonical database as the source of truth.
- Eventually JIT a script-specific classifier/executor from semantic IR when the workload justifies it.

The important architectural point is that none of these ideas require undoing the work already done. The current database is a stable factual layer. Optimization can happen above or beside it.

18. Closing perspective

The most satisfying part of this design is that adding Unicode coverage has become increasingly boring. That is a compliment. A new property usually means an enum, metadata, a parser, one generic generator call, a typed accessor, and an exhaustive test. The storage machinery does not care whether the byte means a bidi class, a joining group, or an Indic position.

That gives me the base I wanted for the rest of the text engine: small enough to understand, complete enough to support real script categorization, and explicit enough that I can validate every layer. The next work belongs above the database - script classifiers, semantic script IR, and eventually a more page-oriented execution model for the hot path.

Appendix A. Quick extension checklist

Binary SET property:

```
source -> parser -> UnicodeCoverageBuilder
        -> shared SET page pool -> coverage root
        -> semantic property record -> runtime membership API
```

VALUE8 property:

```
enum/default -> UnicodeValueProperty8 id -> metadata
source -> parser -> UnicodeValueTable8Builder
        -> shared VALUE8 page pool -> root table
        -> registry -> typed runtime accessor
        -> exhaustive source/runtime comparison
```

Appendix B. Current shaping-facing accessor set

```
database.generalCategory(cp);  
database.combiningClass(cp);  
database.bidiClass(cp);  
database.graphemeClusterBreak(cp);  
database.script(cp);  
database.scriptExtensions(cp);  
  
database.indicConjunctBreak(cp);  
database.indicSyllabicCategory(cp);  
database.indicPositionalCategory(cp);  
  
database.joiningType(cp);  
database.joiningGroup(cp);  
  
database.hangulSyllableType(cp);  
  
database.hasExtendedPictographic();  
database.hasDefaultIgnorableCodePoint();
```